

Diplomado virtual PROGRAMACIÓN EN JAVASCRIPT

Guía didáctica – Módulo 3

```
382     type.pause = function (e) {
383     if (this.paused = true)
384     if (this.$element.find('.next, .prev').length &&
385     this.$element.trigger($.support.transition.end)
386     this.cycle(true)
387     }
388
389     this.interval = clearInterval(this.interval)
390
391     return this
392   }
393
394   Carousel.prototype.next = function () {
395     if (this.sliding) return
396     return this.slide('next')
397   }
398
399   Carousel.prototype.prev = function () {
400     if (this.sliding) return
401     return this.slide('prev')
402   }
403
404   Carousel.prototype.slide = function (type, next) {
405     var $active = this.$element.find('.item.active')
406     var $next = next || this.getItemForDirection(type, $active)
407     var isCycling = this.interval
408     var direction = type == 'next' ? 'left' : 'right'
409     var fallback = type == 'next' ? 'first' : 'last'
410     var that = this
411
412     if (!$next.length) {
413       if (!this.options.wrap) return
414       $next = this.$element.find('.item')[fallback]()
415     }
416
417     if ($next.hasClass('active')) return (this.sliding)
418
419     var relatedTarget = $next[0]
420     var slideEvent = $.Event('slide.bs.carousel', {
421       relatedTarget: relatedTarget,
422       direction: direction
423     })
424     this.$element.trigger(slideEvent)
```



Imagen: Freepik

JS

Guía didáctica

**Arrays,
funciones y
fechas**



Competencia específica

Se espera que, con los temas abordados en la guía didáctica del módulo 3: Arrays, funciones y fechas, el estudiante logre la siguiente competencia específica:

Comprender el uso y la aplicación tanto de los arrays para almacenar información y de las funciones para crear bloques de códigos reutilizables, como de las fechas en JavaScript.



Contenidos temáticos



Imagen: Freepik

Los siguientes son los contenidos temáticos para desarrollar en la guía didáctica del módulo 3: Arrays, funciones y fechas:

1

Arrays

2

Métodos de arrays

3

Funciones

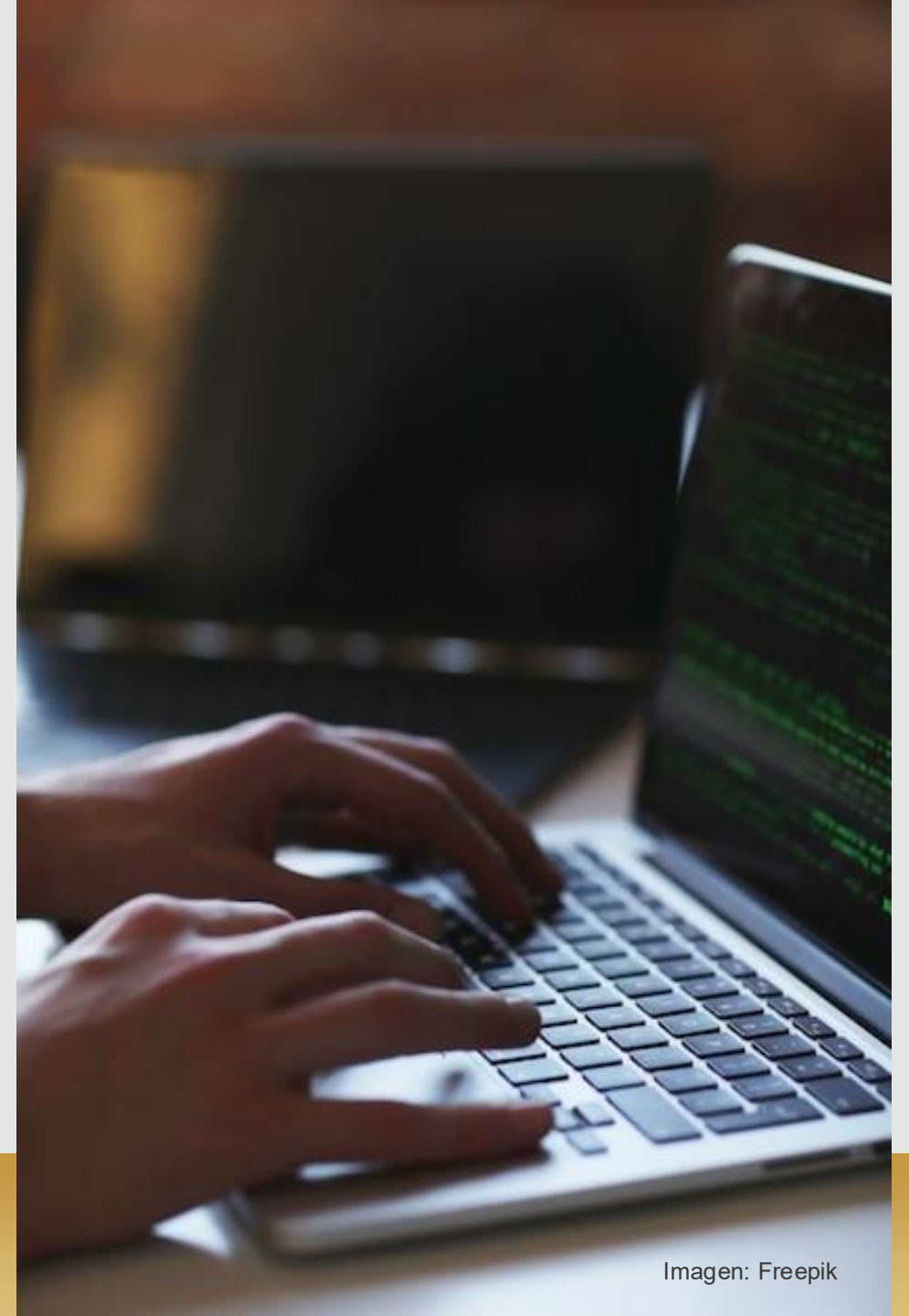
4

Fechas

Activación de saberes previos

Antes de iniciar con el estudio de la guía didáctica, reflexiona sobre las siguientes preguntas:

- Define con tus propias palabras qué es un arrays y qué es una función en JavaScript.
- ¿Cómo se crean, modifican, acceden y eliminan elementos de arrays en JavaScript?
- ¿Qué métodos son frecuentes al utilizar arrays en JavaScript?
- ¿Cómo se crean las funciones en JavaScript?



Tema 1:

Arrays

«La verdad solo se puede encontrar en un lugar: el código».
Robert C. Martín

Arrays

Un **array** es una **colección** o **agrupación** de elementos en una **misma variable**, cada uno de ellos ubicado por la **posición** que ocupa en el **array**.

En algunas ocasiones también se les suelen llamar **arreglos** o **vectores**. Los arrays en JavaScript pueden contener elementos de **cualquier tipo de datos**, incluyendo **números**, **cadenas**, **objetos** y **funciones**.

Los arrays también pueden ser **anidados**, lo que significa que pueden contener otros **arrays** como **elementos**.

Almacenamiento secuencial

Tamaño dinámico

Tipos de datos

Referencias por índices

Métodos

Iteración

Crear

Para crear arrays en JavaScript, podemos utilizar **corchetes** `[]` para definir una lista de elementos, permitiendo almacenar múltiples valores en una sola variable. Por ejemplo:

```
1  const numeros = [1, 2, 3, 4, 5]
2
3  const nombres = ['Juan', 'Pedro', 'María']
4
5  const personas = [
6    { nombre: 'Juan', edad: 20 },
7    { nombre: 'Pedro', edad: 30 },
8    { nombre: 'María', edad: 40 }
9  ]
```

Acceder

Para acceder a los elementos de un array en JavaScript, utilizamos su índice **dentro** de corchetes `[]`, comenzando desde 0.

Por ejemplo, en:

```
1  const nombres = ['Juan', 'Pedro', 'María']
2
3  console.log(nombres[0]) // Juan
4  console.log(nombres[1]) // Pedro
5  console.log(nombres[2]) // María
```


Agregar

Para agregar elementos a un array, podemos usar **índices específicos**. Por ejemplo:

```
1  const nombres = ['Juan', 'Pedro', 'María']
2
3  nombres[3] = 'José'
4  nombres[4] = 'Ana'
5
6  console.log(nombres)
7
8  // ['Juan', 'Pedro', 'María', 'José', 'Ana']
```

Si agregamos un elemento en un índice más alto que la longitud actual del array, los índices intermedios se llenarán con **undefined**.

Modificar

Para modificar elementos de un array, podemos acceder al índice del elemento que deseamos cambiar y **asignarle un nuevo valor**.

Por ejemplo:

```
1  const nombres = ['Juan', 'Pedro', 'María']
2
3  nombres[2] = 'José'
4
5  console.log(nombres)
6
7  // ['Juan', 'Pedro', 'José']
```

Recorrer

Para recorrer los elementos de un array, podemos utilizar el **bucle for**.

Por ejemplo:



```
1  const numeros = [1,2,3,4,5]
2
3  for (let i = 0; i < 5; i++) {
4      console.log(numeros[i])
5  }
```

Arrays en arrays

Los arrays en arrays son arrays anidados y se crean incluyendo un array como **elemento de otro**.

Por ejemplo:



```
1  const numeros =
2      [1,2,3,4,5, [6,7,8,9,10]]
3
4  const nombres =
5      ['Juan', 'Pedro', 'Luis', ['Maria', 'Ana']]
```

Recorrer array en array

Podemos acceder a los elementos anidados con **índices** o usar un **bucle anidado**: el bucle externo recorre los arrays principales, y el interno recorre los elementos de cada subarray.

Por ejemplo:

```
1  const numeros =
2    [
3      [1,2,3],
4      [4,5,6],
5      [7,8,9]
6    ]
7
8  for (let i = 0; i < numeros.length; i++) {
9    for (let j = 0; j < numeros[i].length; j++) {
10      console.log(numeros[i][j])
11    }
12  }
13
```

Destructuring de arrays

En JavaScript, el destructuring de arrays permite extraer valores directamente en variables individuales utilizando **corchetes** [].

Por ejemplo:

```
1  const estudiante =
2    [
3      "Juan",
4      22,
5      true
6    ]
7
8  const [nombre, edad, estatus] = estudiante
9
10 console.log(nombre, edad, estatus) // Juan 22 true
```

Console.table

En JavaScript, podemos usar `console.table()` para mostrar datos en una tabla organizada en la consola del navegador. Es especialmente útil para **visualizar arrays** y **objetos** de manera clara.

Por ejemplo:

```
1  const meses = ['Enero', 'Febrero', 'Marzo']  
2  
3  console.table(meses)
```

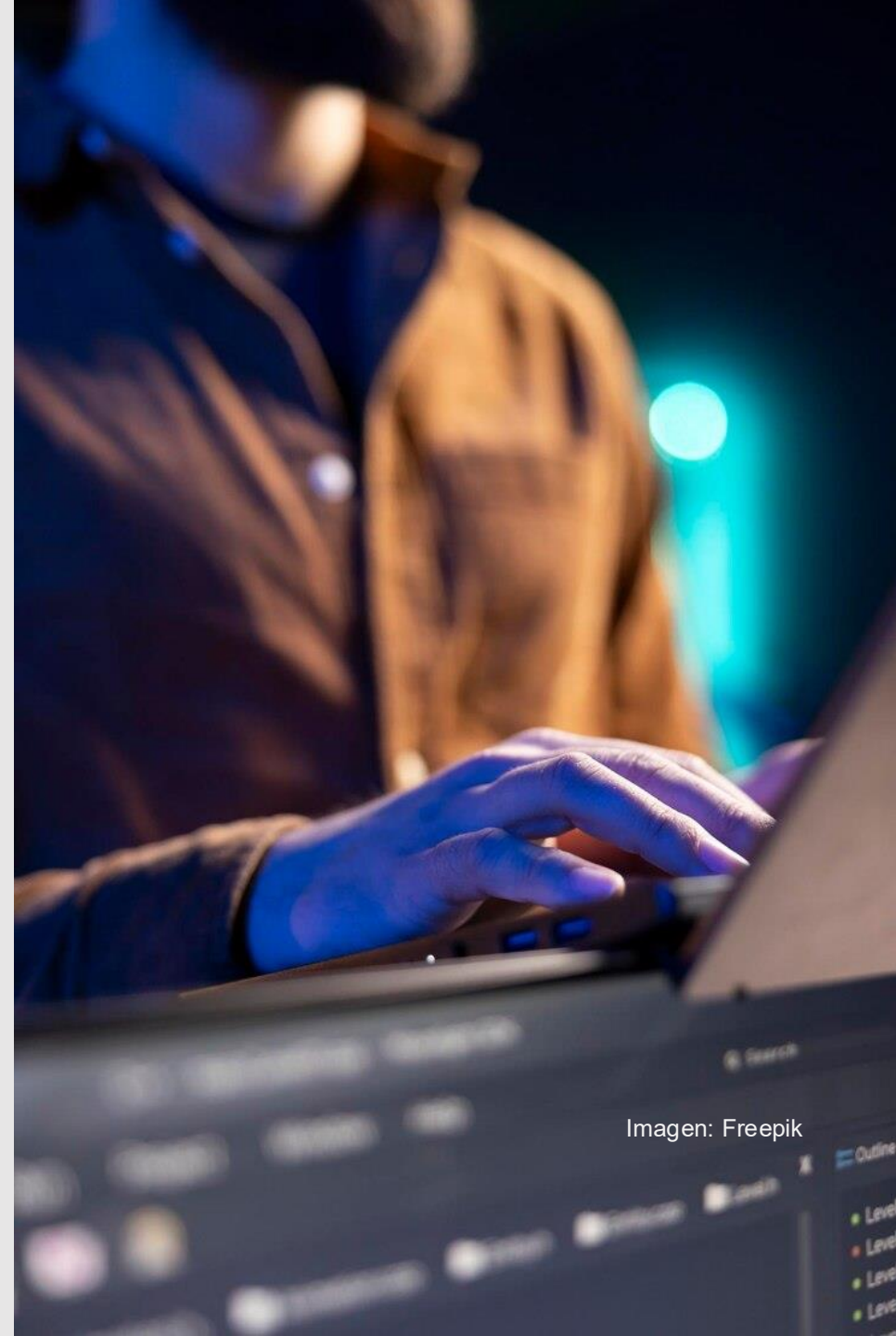


Imagen: Freepik

Tema 2:

Métodos de arrays

«No se puede tener un gran *software* sin un gran equipo».
Jim McCarthy

Métodos de arrays

Son **métodos** que tiene cualquier **variable** que sea de tipo **array** y que permite realizar una operación con todos los **elementos** de dicho **array** (o parte de ellos) para conseguir un **objetivo** concreto, dependiendo del método.

Almacenamiento

Acceso

Iteración

Ordenamiento

Búsqueda

length

Devuelve el número de elementos en un array.

```
1  const emojis = ['😘', '💪', '😓', '🦷']  
2  
3  console.log(emojis.length) // 4
```

push

Agrega uno o más elementos nuevos al **final** de un array y devuelve la nueva longitud.

Este método es mutable, lo que significa que **modifica** el array original.

```
1  const emojis = ['😘', '💪', '😓', '🦷']  
2  
3  emojis.push('👁')  
4  
5  console.log(emojis.length) // 5
```

fill

Rellena los elementos de un array con un valor estático.

```
1  const emojis = ['😓','💪','😞','🦷']
2
3  emojis.fill('😬')
4
5  console.log(emojis)
6
7  // [ '😬', '😬', '😬', '😬', '😬' ]
8
9  emojis.fill('👹', 1, 3)
10
11 console.log(emojis)
12
13 // [ '😬', '👹', '👹', '😬', '😬' ]
```

pop

Elimina el último elemento de un array y devuelve ese elemento.

Este método es mutable.

```
1  const emojis = ['😓','💪','😞','🦷']
2
3  emojis.pop()
4
5  console.log(emojis)
6
7  // [ '😓', '💪', '😞' ]
```

shift

Elimina el **primer elemento** de un array y devuelve ese elemento.

Este método es mutable.

```
1  const emojis = ['😓','💪','😓','🦷']
2
3  emojis.shift()
4
5  console.log(emojis)
6
7  // [ '💪', '😓', '🦷' ]
8
9  console.log(emojis.shift())
10
11 // 💪
12
```

unshift

Agrega nuevos elementos al **principio** de un array y devuelve la nueva longitud.

Este método es mutable.

```
1  const emojis = ['😓','💪','😓','🦷']
2
3  emojis.unshift('👍')
4
5  console.log(emojis)
6
7  // [ '👍', '😓', '💪', '😓', '🦷' ]
8
9  console.log(emojis.unshift('👍'))
10
11 // 5
```

isArray

Comprueba si un elemento es un array.

```
1 const emojis = ['😬', '💪', '😬', '🦷']
2
3 console.log(Array.isArray(emojis)) // true
4
5 console.log(Array.isArray('🦷')) // false
```

forEach

Llama a una función para cada elemento de un array.

```
1 const emojis = ['😬', '💪', '😬', '🦷']
2
3 emojis.forEach(emoji => {
4   console.log(`Emoji: ${emoji}`)
5 })
6
7 // Emoji: 😬
8 // Emoji: 💪
9 // Emoji: 😬
10 // Emoji: 🦷
```

find

Devuelve el valor del primer elemento de un array que cumpla una prueba.

Fórmula:

Const result = nombre del arrays.find()

```
1 const emojis = ['😓', '💪', '😓', '🦷']
2
3 const result =
4   emojis.find(emoji => emoji === '🦷')
5
6 console.log(`Emoji Encontrado: ${result}`)
7
8 // Emoji Encontrado: 🦷
```

filter

Crea un nuevo array con cada elemento de un array que cumpla una prueba.

Fórmula:

Const result= nombre del arrays.filter()

```
1 const emojis = ['😓', '💪', '😓', '🦷']
2
3 const result =
4   emojis.filter(emoji => emoji !== '🦷')
5
6 console.log(result)
7
8 // ['😓', '💪', '😓']
```


concat

Une arrays y **devuelve** un array nuevo con los arrays unidos.

```
1 const emojisPack = ['😬', '💪', '😬', '🦷']
2
3 const emojiNewPack = ['😱', '❤️', '👻', '📧']
4
5 const emojis = emojisPack.concat(emojiNewPack)
6
7 // [ '😬', '💪', '😬', '🦷', '😱', '❤️', '👻', '📧' ]
```

```
1 const emojisPack = ['😬', '💪', '😬', '🦷']
2
3 const emojiNewPack = ['😱', '❤️', '👻', '📧']
4
5 const emojis = [ ... emojisPack, ... emojiNewPack ]
6
7 // [ '😬', '💪', '😬', '🦷', '😱', '❤️', '👻', '📧' ]
```

findIndex

Devuelve el **índice del primer elemento** de un array que cumpla una prueba.

```
1 const emojis = ['😬', '💪', '😬', '🦷']
2
3 const index = emojis.indexOf('😬')
4
5 console.log(index) // 2
```

map

Crea un nuevo array con el resultado de **llamar** a una función para cada elemento de un array.

```
1 const emojis = ['😬', '💪', '😬', '🦷']
2
3 const result = emojis.map(emoji => {
4   return `${emoji}✅`
5 })
6
7 console.log(result)
8
9 // [ '😬✅', '💪✅', '😬✅', '🦷✅' ]
```

join

Une todos los elementos de un array en una cadena o **string**, separados por un delimitador.

```
1 const emojis = ['😬', '💪', '😬', '🦷']
2
3 console.log(emojis.join(''))
4
5 // 😬💪😬🦷
6
7 console.log(emojis.join(' '))
8
9 // 😬 💪 😬 🦷
10
11 console.log(emojis.join('👁👁'))
12
13 // 😬👁👁💪👁👁😬👁👁🦷
14
15 console.log(emojis.join('-'))
16
17 // 😬-💪-😬-🦷
```

reduce

Reduce los valores de un array a un único valor (de izquierda a derecha).



```
1  const emojis = ['😓', '💪', '😓', '🦷']
2
3  emojis.reduce((acc, emoji) => {
4    console.log(acc + emoji)
5    return acc + emoji
6  })
7
8  // 😓
9  // 😓💪
10 // 😓💪😓
11 // 😓💪😓🦷
```

slice

Selecciona una parte de un array y devuelve el nuevo array.



```
1  const emojis = ['😓', '💪', '😓', '🦷']
2
3  let resultUno = emojis.slice(1, 3)
4
5  console.log(resultUno)
6
7  // [ '💪', '😓' ]
8
9  let resultDos = emojis.slice(2)
10
11 console.log(resultDos)
12
13 // [ '😓', '🦷' ]
```

some

Comprueba si alguno de los elementos de un array cumple una prueba.

Fórmula:

Console. Log(nombre del arrays.some())

```
1 const emojis = ['😬', '💪', '😞', '🦷']
2
3 console.log(
4   emojis.some(emoji => emoji === '🦷')
5 )
6
7 // true
```

sort

Ordena los elementos de un array. **Este método es mutable.**

Fórmula:

Const arraysOrdenado= nombre del arrays.sort()

```
1 const emojis = ['😬', '💪', '😞', '🦷']
2
3 const emojisOrdenados = emojis.sort()
4
5 console.log(emojisOrdenados)
6
7 // ['😞', '💪', '😬', '🦷']
8
9 const numeros = [5, 1, 8, 2, 4]
10
11 const numerosOrdenados = numeros.sort()
12
13 console.log(numerosOrdenados)
14
15 // [1, 2, 4, 5, 8]
```

splice

Agrega/elimina elementos de un array. **Este método es mutable.**

Fórmula:

Console.log(nombre del arrays.splice(indice 1, indice 2))

```
1 const emojis = ['😬', '👁', '💪', '😬', '🦷']
2
3 console.log(emojis.splice(0, 2))
4
5 // ['😬', '👁']
6
7 console.log(emojis)
8
9 // ['💪', '😬', '🦷']
10
11 console.log(emojis.splice(0, 2, '👄', '👄'))
12
13 // ['💪', '😬']
14
15 console.log(emojis)
16
17 // ['👄', '👄', '🦷']
```

toString

Convierte un array en una cadena y devuelve el resultado.

Fórmula:

Console.log(nombre del arrays.toString())

```
1 const emojis = ['😬', '👁', '💪', '😬', '🦷']
2
3 console.log(emojis.toString())
4
5 // 😬,👁,💪,😬,🦷
```

every

Comprueba si todos los elementos de un array cumplen una **condición**.

Por ejemplo:

```
1 const emojis = ['😓','😓','💪','😓','😓']
2
3 console.log(emojis.every(emoji => emoji === '😓'))
4
5 // false
6
7 const newEmojis = ['😓','😓','😓','😓','😓']
8
9 console.log(newEmojis.every(emoji => emoji === '😓'))
10
11 // true
```

reverse

Invierte todos los elementos de un array. **Este método es mutable**.

Fórmula:

Console.log(nombre del arrays.reverse())

```
1 const emojis = ['😓','👁','💪','😓','🦷']
2
3 console.log(emojis.reverse())
4
5 // [ '🦷', '😓', '💪', '👁', '😓' ]
```



Practica lo aprendido

Pon en práctica lo visto acerca de arrays resolviendo los siguientes ejercicios:

Ejercicio 1

Dado un array de números, verifica si los elementos están ordenados en orden ascendente.

Devuelve un mensaje indicando si el array está ordenado o no.

Ejercicio 2

Dado un número n , crea un array con todos los números impares desde 1 hasta n (incluyendo n si es impar).

Ejercicio 3

Dado un array, intercambia el primero y el último elemento.

Si el array tiene solo un elemento o está vacío, no hagas nada.

Ejercicio 4

Dado un array, crea una copia exacta de este sin utilizar ningún método de arrays.

Usa un bucle para copiar cada elemento a un nuevo array.

Practica lo aprendido



Imagen: Freepik

Ejercicio 5

Dado un array de números, calcula la suma de todos los elementos en el array. Utiliza un bucle para sumar cada elemento.

Ejercicio 6

Dado un array de números, encuentra el número más grande en el array sin utilizar ningún método predefinido.

Ejercicio 7

Dado un array y un número, cuenta cuántas veces aparece ese número en el array.

Ejercicio 8

Dado un array de números, cuenta cuántos elementos son pares.

Ejercicio 9

Dado un array de números, encuentra el número más pequeño en el array.

Ejercicio 10

Dado un array, invierte sus elementos. Modifica el array original y utiliza un bucle.

Tema 3:

Funciones

«Controlar la complejidad es la esencia de la programación».
Brian Kernigan

Funciones

Las funciones en JavaScript son bloques de código que se pueden **reutilizar** para realizar una tarea específica. Las funciones pueden tomar **argumentos** de entrada y producir un **resultado** de salida, o pueden no tomar ningún argumento y simplemente realizar una tarea.



```
1 function holaMundo() {  
2     console.log('Hola mundo')  
3 }  
4  
5 holaMundo() // Hola mundo  
6 holaMundo() // Hola mundo
```

Funciones != Métodos

En programación, tanto las funciones como los métodos son bloques de código que se utilizan para realizar una tarea específica. Sin embargo, existen algunas diferencias entre ambas:

Definición

Invocación

Acceso

Propósito

Sintaxis

Definición: las funciones son bloques de código independientes que pueden recibir argumentos y devolver un valor. Los métodos, por otro lado, son funciones que se definen dentro de una clase o un objeto.

Invocación: las funciones se invocan directamente desde el código, mientras que los métodos se invocan a través de un objeto o una instancia de una clase.

Acceso a propiedades y métodos: las funciones pueden acceder a variables y funciones definidas en su ámbito, mientras que los métodos pueden acceder a las propiedades y métodos del objeto o la instancia de la clase en la que están definidos.

Funciones != Métodos

Propósito: las funciones se utilizan para realizar una tarea específica y pueden ser reutilizadas en diferentes partes del código. Los métodos, por otro lado, se utilizan para representar el comportamiento de un objeto y están diseñados para interactuar con las propiedades y los métodos del objeto.

Sintaxis: las funciones se definen utilizando la palabra clave **function** seguida del nombre de la función y los parámetros entre paréntesis. Los métodos se definen dentro de una clase o un objeto y se acceden utilizando la notación de punto.

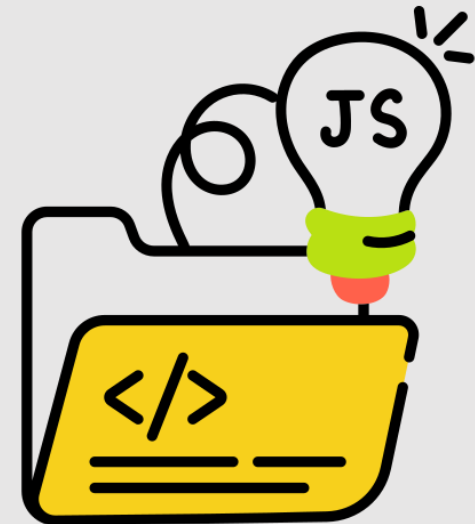


Imagen: Flaticon



Características de las funciones

Las funciones cuentan con algunas características que permiten y potencia su uso y alcance, estas son algunas de ellas.

Parámetros

Argumentos

Retornos

Parámetros

Los **parámetros** son los nombres que se definen en la **declaración** de una función y que se utilizan para **recibir valores**. Es decir, son los «huecos» que se crean en la definición de la función para que se puedan pasar valores cuando se llame la **función**.

```
1  function saludar(nombre) {  
2      console.log(`Hola ${nombre}`)  
3  }  
4  
5  saludar('Juan') // Hola Juan  
6  
7  function sumar(numero1, numero2) {  
8      console.log(numero1 + numero2)  
9  }  
10  
11 sumar(2, 3) // 5
```

Es posible que en algunos casos queramos que ciertos parámetros tengan un valor sin necesidad de escribirlos en la ejecución. Es lo que se llama un **valor por defecto**.

Argumentos

Los **argumentos** son los valores **reales** que se pasan a una función cuando se le **llama**. Es decir, son los valores que se colocan en los «huecos» que se crearon en la definición de la **función**.



```
1  function saludar(nombre) {  
2      console.log(`Hola ${nombre}`)  
3  }  
4  
5  saludar('Juan') // Hola Juan  
6  
7  function sumar(numero1, numero2) {  
8      console.log(numero1 + numero2)  
9  }  
10  
11  sumar(2, 3) // 5
```

Return (retornos)

En JavaScript, la palabra reservada **return** se utiliza para **devolver** un **valor** desde una función. Cuando se ejecuta una instrucción **return** en una función, se interrumpe inmediatamente la ejecución de la función y se devuelve el **valor especificado**.



```
1  function sumar(a, b) {  
2      return a + b  
3  }  
4  
5  function restar(a, b) {  
6      return a - b  
7  }  
8  
9  function multiplicar(a, b) {  
10     return a * b  
11 }
```




Tipos de funciones

Existen varios **tipos** de funciones definidos por su forma de declarar y funcionar. Estos tipos son:

Declarativas

Anónimas

Autoejecutables

Flechas

Declarativas

Es la forma más **común** de declarar una función en JavaScript. Se define con la palabra clave **function** seguida del nombre de la función, paréntesis y llaves que delimitan el cuerpo de la función.



```
1  function mostrarMensaje() {  
2      console.log('Hola mundo')  
3  }  
4  
5  function compararNumeros(a, b) {  
6  
7      if (a > b) {  
8          console.log(a + ' es mayor que ' + b)  
9      } else if (a < b) {  
10         console.log(a + ' es menor que ' + b)  
11     } else {  
12         console.log('Los números son iguales')  
13     }  
14 }
```

Anónimas

Una **función anónima** es una función que no tiene un nombre **definido**. En lugar de ello, se define como una **expresión** y se asigna a una **variable** o se pasa como argumento a otra función.

Las funciones **anónimas** son útiles en situaciones donde se necesita una **función temporal** o de una sola vez, ya que no es necesario definirla con un nombre separado y luego llamarla. En su lugar, se pueden **definir** y **usar** en el mismo lugar.



```
1  const comparacion = function(a, b) {  
2  
3      if (a > b) {  
4          console.log(a + ' es mayor que ' + b)  
5      } else if (a < b) {  
6          console.log(a + ' es menor que ' + b)  
7      } else {  
8          console.log('Los números son iguales')  
9      }  
10 }  
11  
12 comparacion(5, 5)  
13  
14 // Los números son iguales
```

Autoejecutables

Son funciones que se ejecutan **inmediatamente** después de su definición.

```
1  (function() {  
2      let contador = 0  
3  
4      const incrementar = () => {  
5          contador++  
6          console.log(contador)  
7      }  
8  
9      const decrementar = () => {  
10         contador--  
11         console.log(contador)  
12     }  
13  
14     incrementar() // 1  
15     incrementar() // 2  
16     incrementar() // 3  
17     decrementar() // 2  
18 })()
```

Flechas

En JavaScript, las funciones de flecha (**arrow functions**) son una forma más **concisa** de escribir funciones anónimas.

En lugar de la palabra clave **function**, las funciones de **flecha** se definen utilizando una sintaxis de flecha (**=>**).



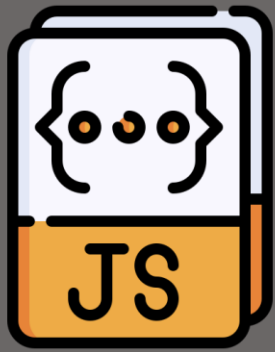
```
1  const suma = (a, b) => a + b
2
3  console.log(suma(2, 3)) // 5
4
5  const resta = (a, b) => a - b
6
7  console.log(resta(2, 3)) // -1
8
9  const multiplicacion = (a, b) => {
10     return a * b
11 }
12
13 console.log(multiplicacion(2, 3)) // 6
14
15 const division = (a, b) => {
16     console.log(a / b)
17 }
18
19 division(2, 3) // 0.6666666666666666
```

Callbacks

Los **callbacks** no son más que un tipo de funciones que se pasan por **parámetro** a otras **funciones**. El objetivo de esto es tener una forma más legible de escribir funciones, más cómoda y flexible para reutilizarlas, y además entra bastante en consonancia con el concepto de **asincronía** de JavaScript, como veremos más adelante.

Los **callbacks** se utilizan comúnmente para realizar operaciones **asíncronas** en JavaScript.

```
1  const datos = (callback) => {
2
3      callback()
4
5      console.log({
6          nombre: 'Juan',
7          edad: 23,
8      })
9  }
10
11  const saludar = () => {
12      console.log('Hola mundo')
13  }
14
15  datos(saludar)
```



Practica lo aprendido

Para practicar lo visto sobre funciones, resuelve los siguientes ejercicios:

Ejercicio 1

Crea una función `calcularAreaRectangulo` que tome el ancho y la altura de un rectángulo y devuelva su área.

Ejercicio 2

Crea una función `celsiusAFahrenheit` que tome una temperatura en grados Celsius y la convierta a Fahrenheit.

La fórmula es: $F = C * 9/5 + 32$.

Ejercicio 3

Crea una función `esPar` que tome un número como parámetro y devuelva `true` si el número es par, o `false` si es impar.

Ejercicio 4

Crea una función `contarVocales` que tome una cadena como parámetro y devuelva el número de vocales (a, e, i, o, u) en la cadena.

Practica lo aprendido



Ejercicio 5

Crea una función `numeroAleatorioEnRango` que tome dos números como argumentos (mínimo y máximo) y devuelva un número aleatorio entre esos dos valores, inclusive.

Ejercicio 6

Crea una función `revertirCadena` que tome una cadena como parámetro y devuelva la cadena invertida.

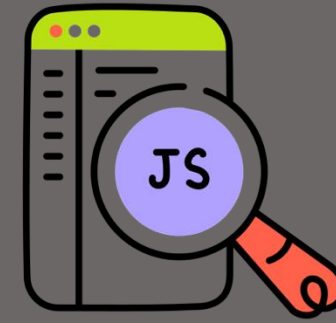
Ejercicio 7

Crea una función `esPalindromo` que tome una palabra y devuelva `true` si es un palíndromo (se lee igual de adelante hacia atrás), o `false` si no lo es.

Ejercicio 8

Crea una función `sumaArray` que tome un array de números y devuelva la suma de todos sus elementos.

Practica lo aprendido



Ejercicio 9

Crea una función `esLarga` que tome una palabra como parámetro y devuelva `true` si la palabra tiene más de 5 letras, o `false` en caso contrario.

Ejercicio 10

Crea una función `promedio` que tome un arreglo de números y devuelva el promedio de esos números.

Ejercicio 11

Crea una función `filtrarPositivos` que reciba un arreglo de números y devuelva un nuevo arreglo solo con los números positivos.

Ejercicio 12

Crea una función `contienePalabra` que tome dos parámetros: una frase y una palabra, y devuelva `true` si la palabra se encuentra en la frase, o `false` si no.

Tema 4:

Fechas

«El arte desafía a la tecnología, y la tecnología inspira al arte».
Jhon Lasseter

Fechas

En JavaScript, las **fechas** se manejan a través del objeto integrado **Date**, que proporciona **métodos** para **crear**, **manipular** y **formatear** fechas y horas.

Aunque **Date** tiene sus limitaciones, es muy versátil y permite trabajar tanto con fechas **actuales** como con valores específicos en el **pasado** o el **futuro**.



```
1  const fecha = new Date()  
2  
3  console.log(fecha) // 2024-11-04T20:00:00.000Z  
4  
5  fecha.setDate(10)  
6  fecha.setMonth(4)  
7  fecha.setFullYear(2024)  
8  
9  console.log(fecha) // 2024-05-10T20:00:00.000Z
```

Crear fecha

El objeto **Date** permite crear fechas de diferentes formas, siendo la más común la fecha actual.

Se puede crear una fecha específica usando el constructor **Date** con los parámetros necesarios.

Una fecha en texto puede ser también creada por medio de **Date**. Los milisegundos también suele ser otra opción muy viable y exacta.

```
1  const fecha = new Date()
2
3  console.log(fecha) // 2024-11-04T15:30:00.000Z
4
5  const fechaEspecifica = new Date(1999, 7, 4, 15, 30)
6
7  console.log(fechaEspecifica) // 1999-08-04T15:30:00.000Z
8
9  const fechaTexto = new Date('1999-08-04T15:30:00')
10
11 console.log(fechaTexto) // 1999-08-04T15:30:00.000Z
12
13 const fechaMilisegundos = new Date(1630753889000)
14
15 console.log(fechaMilisegundos) // 2021-09-04T15:31:29.000Z
16
```

Métodos

Obtención de fecha

Por medio de **get** se pueden obtener diferentes componentes de las fechas de forma individual (Developer Mozilla, 2025).

```
1  const fecha = new Date()
2
3  console.log(fecha) // 2024-11-01T15:30:00.000Z
4
5  console.log(fecha.getDate()) // 1
6  console.log(fecha.getMonth()) // 10
7  console.log(fecha.getFullYear()) // 2024
8  console.log(fecha.getHours()) // 15
9  console.log(fecha.getMinutes()) // 30
10 console.log(fecha.getSeconds()) // 0
11
12 // Los meses inician en 0, por lo que 10 es noviembre
```

Establecer fecha

Al igual que get, **set** permite establecer valores para los parámetros de forma individual de una fecha (Developer Mozilla, 2025).

```
1  const fecha = new Date()
2
3  console.log(fecha) // 2024-11-01T15:30:00.000Z
4
5  fecha.setDate(7)
6  fecha.setMonth(8)
7  fecha.setFullYear(2021)
8  fecha.setHours(10)
9  fecha.setMinutes(30)
10 fecha.setSeconds(0)
11
12 console.log(fecha) // 2021-09-07T10:30:00.000Z
```

Operaciones

Comparar fechas

En JavaScript, podemos comparar fechas usando operadores (<, >, <=, >=, ==, ===) o restando dos fechas para obtener la diferencia en milisegundos (Developer Mozilla, 2025).

```
1 const fecha1 = new Date("2024-10-12")
2 const fecha2 = new Date("2022-08-05")
3
4 console.log(fecha1 > fecha2) // true
5 console.log(fecha1 < fecha2) // false
6 console.log(fecha1 === fecha2) // false
7 console.log(fecha1 !== fecha2) // true
8 console.log(fecha1 - fecha2) // 77760000000
```

Sumar y restar

Para sumar o restar componentes, podemos obtener el componente actual y modificarlo usando **set** (Developer Mozilla, 2025).

```
1 const fecha = new Date("2024-01-01")
2
3 console.log(fecha) // 2024-01-01T00:00:00.000Z
4
5 fecha.setDate(fecha.getDate() + 7) // Añadir 7 días
6
7 console.log(fecha) // 2024-01-08T00:00:00.000Z
8
9 fecha.setMonth(fecha.getMonth() + 1) // Añadir 1 mes
10
11 console.log(fecha) // 2024-02-08T00:00:00.000Z
```

Practica lo aprendido

Pon en práctica lo visto acerca de fechas, realizando los siguientes ejercicios:

Ejercicio 1

Crea una función `calcularEdad` que reciba la fecha de nacimiento de una persona (en formato YYYY-MM-DD) y calcule su edad actual en años.

Ejercicio 2

Crea una función `nombreDiaSemana` que reciba una fecha en formato YYYY-MM-DD y devuelva el nombre del día de la semana (por ejemplo, «lunes»).

Ejercicio 3

Escribe una función `diasHastaCumpleanos` que reciba una fecha de nacimiento y devuelva cuántos días faltan hasta el próximo cumpleaños.

Ejercicio 4

Crea una función `esFinDeSemana` que reciba una fecha y devuelva `true` si la fecha cae en sábado o domingo, o `false` en caso contrario.

Ejercicio 5

Crea una función `esBisiesto` que reciba un año y devuelva `true` si el año es bisiesto, o `false` si no lo es.

Referencia bibliográfica

Developer Mozilla. (2025). *Fecha*. Developer Mozilla. https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Date

Esta guía fue elaborada para ser utilizada con fines didácticos como material de consulta de los participantes en el diplomado virtual en PROGRAMACIÓN EN JAVASCRIPT del Politécnico de Colombia, y solo podrá ser reproducida con esos fines. Por lo tanto, se agradece a los usuarios referirla en los escritos donde se utilice la información que aquí se presenta.

GUÍA DIDÁCTICA 3

M3-DV114-GU03

MÓDULO 3: ARRAYS, FUNCIONES Y FECHAS

© DERECHOS RESERVADOS - POLITÉCNICO DE
COLOMBIA, 2025
Medellín, Colombia

Proceso: Gestión Académica Virtual

Realización del texto: Diego Alejandro Palacio, docente

Revisión del texto: Comité de Revisión

Diseño: Comunicaciones

Editado por el Politécnico de Colombia

